

Emuqu — Blind-Audit Convergence: A Method for Directing AI to Produce Converged Software Designs

Memory-blind auditors, an exclusion list carried inside the audited document, adjudication against one goal statement, and convergence judged by the character of what is left

Technical White Paper • v1.1 • 8 October 2026

Chris Sharp • github.com/chrissharp80

Describes the design and review method used to build Emuqu (formerly Flow HRV, then Flow Recovery), as practiced from February 2025 to October 2026. Emuqu is a consumer wellness app, not a medical device.

Cite as: Sharp C. Emuqu — Blind-Audit Convergence: A Method for Directing AI to Produce Converged Software Designs. Technical White Paper v1.1, 8 October 2026. github.com/chrissharp80/emuqu_docs/whitepapers/05-blind-audit-convergence-method.md.

Executive Summary

Emuqu, an iOS app that turns overnight beat-to-beat (RR) intervals from a chest strap into a daily recovery score, was designed and written by artificial intelligence (AI) models under the author's direction; he wrote no code by hand. This paper describes how its design was produced, which the author regards as the most important part of the work.

A single model asked for a design is confident, plausible and wrong in places it cannot see, and cannot check itself. A second model always finds something, and acting on every finding makes the design grow without end. Blind-audit convergence handles both with fixed rules:

- **Roles.** A **Creator** model writes a complete design. **Auditors**, fresh memory-off instances, sometimes from another model family, read only the document. The **director** (the author) owns the goal statement and decides when the design is done.
- **The document carries its own audit contract:** the design, an **AUDIT INSTRUCTIONS** section and a **DO NOT FLAG** section, the *exclusion list* of settled findings with reasons (the author's "do-not-ding list").
- **Fixed adjudication.** Accept what serves the goal; reject scope creep, bloat and security theater; write every rejection into DO NOT FLAG.
- **No iteration cues.** Version numbers and words such as "corrected" are removed.
- **Convergence by character, not count.** The loop runs for tens of rounds and stops when what remains is cosmetic or improbable.
- **Then stack and code.** A fresh instance chooses the stack; code follows a refactor specification and an 18-pass audit graded P0 (blocks release) to P3 (nit), and every finding is fixed.

The loop was automated in DesignHub, the author's Python tool, on 2026-01-21. The author developed the method independently and did not learn it from anyone. A prior-art search on 2026-10-08 found many parts in earlier work, credited in Section 7, but no earlier publication of the mechanisms stated there, chiefly the exclusion list carried in the document for stateless auditors, and the full pipeline. The paper does not claim to be first.

Dated records (Section 9) show the audit loop in use by 2025-02-22; the worked example (Section 4) ends with a converged design saved on 2026-01-08 21:06 US Central Standard Time (CST) and code four days later. The result is verified by use, beta testers and continuous integration (CI) gates (Section 2.13).

1. The Problem

A capable artificial-intelligence (AI) model asked to design an app returns a long, well-organized document whose failures are not visible on the page. Six recur, and each defeats a simpler workflow.

1.1 Confident errors

Large language models (LLMs) produce fluent statements that are false, and they phrase them with the same confidence as true ones [1]. In a design such an error looks like a fact: a sensor "cannot" record what it can, or a threshold is too strict for some users. Nothing in the text marks it out.

1.2 Self-evaluation bias

The obvious fix is to ask the model to check its own work. Models judging text tend to favor text they produced themselves [2][3], and without outside feedback self-correction does not reliably improve reasoning and can make it worse [4]. Models also tend to agree with the person they are talking to [5]. A Creator asked "is this right?" in the conversation where it wrote the design is a weak reviewer.

1.3 Drift in long conversations

A design revised many times in one conversation accumulates history. Models use long contexts unevenly [6] and lose track of earlier constraints over many turns [7], so requirements drop out silently and rejected ideas return.

1.4 Endless re-raised findings

A fresh reviewer avoids self-bias and drift but knows nothing of decisions already made, so it raises them again. The author wrote on 2025-09-13, about a plan that had already been audited dozens of times: "... on every audit ... like 50x already." A stateless reviewer that re-raises settled points every round produces a loop that cannot end.

1.5 Feature creep

Every audit suggests additions: another backup tier, another security layer, another option. Each is reasonable alone; together they turn a focused design into something else. Reviewers asked to "find problems" will always find some, and Anthropic's current guidance for its own coding agent notes that chasing every reviewer gap causes over-engineering [8].

1.6 No stopping rule

Iterative-refinement methods mostly stop after a fixed number of rounds or on a model-emitted signal, and report that gains level off within a few iterations [9]. A full product design needs more rounds than that. It also needs a way to tell "the reviewers have run out of real problems" from "this round found nothing", because a reviewer that always reports something never reaches zero.

The method in Section 2 is built so that each of these failures has a specific counter (Section 3).

2. The Method

2.1 Roles

Table 2.1. Roles in blind-audit convergence.

Role	Who	Sees	Decides
Director	The author	Everything	The goal statement; when the design has converged; final disposition of anything contested
Creator	One model instance, kept for the whole design	The goal statement, its own design, each audit as returned	How to revise; which findings to accept; what goes into DO NOT FLAG
Auditor	A new instance for every audit, memory off; sometimes a different model family	The current design document, nothing else	What is wrong with the document as written
Stack chooser	A new instance after convergence	The converged design and the product goals	The implementation stack, after researching current options
Implementer	A coding agent	The design, the refactor specification, the codebase	The code
Code auditor	A fresh instance per audit	The codebase, the refactor specification, the audit specification	Graded findings with exact fixes

Creators have been ChatGPT and later Claude; auditors fresh ChatGPT conversations, memory-off Claude instances, and at times Gemini, DeepSeek and Grok. Of one run the author wrote ($\leq 2026-02-05$): "first 30+ rounds using one chatgpt creator and blind chatgpt instances. then i switched to opus 4.5 as the creator still using gpt5.2 as the blind auditor."

2.2 The design artifact

The unit that moves through the loop is a single document with three parts. DesignHub's Creator prompt fixes the format:

```

## DESIGN
[the design/plan]

## AUDIT INSTRUCTIONS
[tell the auditor what to check for]

## DO NOT FLAG
[things the auditor should ignore]

```

DESIGN is the complete design, written out in full every round, never a diff. Section 4.7 shows why.

AUDIT INSTRUCTIONS tells the auditor what to check: conformance to the goal, factual claims, contradictions, missing specifications, real defects only. The Creator writes it, so the document, not whoever opens the next auditor, sets each audit's scope.

DO NOT FLAG is the exclusion list: findings that have been considered and settled, each with a one-line reason. It holds three kinds of entry:

1. **Rejected findings:** things an auditor proposed that do not serve the goal ("NO secondary PostgreSQL instance ... Why? Not needed with WAL + PITR and multiple cloud/local backups", where WAL + PITR is write-ahead logging with point-in-time recovery).
2. **Verified facts:** claims an auditor doubted that were then researched and confirmed, with the source ("H10 capacity verified (95,000 RR intervals = 20+ hours) — No testing needed", where the H10 is the Polar chest strap and RR intervals are beat-to-beat intervals).
3. **Locked decisions:** deliberate design choices with their rationale, so that an auditor who would have chosen differently does not reopen them (for example, "Archive hash scope: CSV only").

The list only grows, unless the director reopens a decision.

2.3 Blindness

An auditor sees the document and nothing else. In practice:

1. Open a new conversation or application programming interface (API) call for every audit. Never reuse an auditor.
2. Turn memory off, or use an API call with no history.
3. Paste the document exactly as the Creator wrote it. DesignHub's rule is "Creator output = Audit input (verbatim, no hub modifications)", and the auditor's message contains "exactly what Creator output, nothing else."
4. Give the auditor a minimal prompt. In manual runs this was "Please audit this plan" or "please audit this design." In DesignHub the auditor's system prompt is one line: "You are an AUDITOR. Follow the audit instructions in the design."
5. Never tell the auditor about earlier rounds, earlier auditors, the director's opinion, or who wrote the design.

2.4 Hiding iteration cues

An auditor that can tell a document has been heavily revised is likely to review it differently (Section 3.7), so the Creator removes every sign that the process is ongoing:

- no version numbers in the title or body;
- no "changes from the previous version" section;
- no words such as "corrected", "revised", "fixed" or "now" that imply an earlier state;
- DO NOT FLAG entries written as standing decisions ("not used", "accepted for this scope"), not as a history of what auditors said.

In the author's words (2026-08-10 article [10]): "I also made sure it didn't add version numbers or anything that would alert the next round of auditors that this was an ongoing process." A later instruction ($\leq$ 2026-06-06) specifies that the word "corrected" never appears in design documents.

2.5 Adjudication rules

Every audit returns to the Creator wrapped in the same instructions. DesignHub's wrapper (Appendix A.4) is the shortest form: "Accept changes that conform to the original spec. Reject scope creep, bloat, security theater, unnecessary additions. If you reject something, add it to 'DO NOT FLAG'." The rules in full:

1. **The goal statement is the only authority.** A later DesignHub version puts it as "THE IDEA STATEMENT IS YOUR ONLY AUTHORITY." A finding is accepted only if fixing it makes the design better at what the goal statement says the product is for.
2. **Accept real defects.** Accept contradictions, wrong facts, missing specifications, unhandled failure modes and anything that would make the product fail at its goal.
3. **Reject scope creep.** Reject features, options or integrations the goal does not ask for, however reasonable.
4. **Reject bloat.** Reject redundant layers, extra infrastructure and generality the product will not use.
5. **Reject security theater.** Reject protections that address no real threat in this product's deployment. The author's reply to one such code-audit finding (2026-02-23): "this runs on iphone. stores to icloud. what needs to be encrypted about that?"
6. **Research before ruling.** When an auditor disputes a fact, look it up. If the auditor is right, fix the design. If it is wrong, write the verified fact and its source into DO NOT FLAG so no later auditor raises it again.
7. **Write every rejection back.** Each rejected finding becomes a DO NOT FLAG entry with its reason, in the same revision.
8. **Output only the document.** The Creator returns the complete revised design in the three-part format, with no commentary, so it can go straight to the next auditor.

Some findings are real but not worth acting on now. These get one of three dispositions, which the worked example records in its own words: "Edge cases caught by acceptance tests", "Acceptable simplifications documented as such", or "Post-ship improvements". Each disposition is written into the document, so it too is settled for later auditors.

In the manual loop the wrapper was the author's own message, "Here are the audit results for your design," with a warning to the Creator "to be leery of feature creep, to take only the items from the audits that were relevant to the

mission statement and to the app that we had already agreed on building, and to create a do-not-ding list" (2026-08-10 article [10]).

2.6 The loop

A *round* is one Creator revision plus the audits of it; "cycle" and "iteration" in quoted material mean the same.

1. The director writes the goal statement: what the product is for, who uses it and what it must never do. Where the domain needs it, deep research comes first and is gathered into a brief.
2. The Creator writes the first complete design in the three-part format.
3. The director opens one or more blind auditors (one per round in the 2025 practice; four per round in the author's account of the Emuqu design) and gives each the document verbatim.
4. Each audit goes back to the Creator under the rules in Section 2.5.
5. The Creator returns the complete revised document, with DO NOT FLAG updated and iteration cues removed.
6. The director reads the remaining findings and judges convergence (Section 2.7). If the design has not converged, return to step 3 with new auditors.
7. After convergence, run a final audit from a model family other than the Creator's (Section 2.8; a fixed step since September 2026, Section 5.4).

The loop is long by design. On 2025-04-27 the author wrote: "It takes roughly 40 instances before there's finally a convergence ... I didn't get my number from researching some books, man. I did it from actually using you." In the 2025 practice each round used one new auditor instance ("give that plan to a new instance to audit ... another new instance is consulted"), so instances and rounds coincide. His later account of the Emuqu design (2026-08-10) is 40 to 50 rounds and around 200 audits. Both are his own counts, the first written at the time and the second recalled; no round-by-round log was kept.

2.7 Convergence and stopping

The director judges convergence by looking at what the auditors are still finding, not by how many findings there are.

Table 2.2. What auditors find in each phase of the loop.

Phase	What auditors find	What it means
Early	Wrong facts, contradictions, missing subsystems, infeasible requirements	The design is still changing in substance
Middle	Edge cases, off-by-one rules, unclear ownership of data, test gaps	The structure is settled; details are being fixed
Converged	Cosmetic preferences, test-code bugs, memory-management nits, improbable attack scenarios, findings already on DO NOT FLAG	The auditors have run out of design defects

Two statements mark the converged phase. In the worked example, the Creator's assessment at the author's request, on or before 2026-01-08, was "The auditor is now finding test bugs and memory leaks, not design flaws. That's the

signal that the design is done." In the author's general account: "I know I've reached convergence when the problems that I'm being told to solve are things like font size and some really weird security improbability."

DesignHub's specification (Appendix A.1) states the same rule as three stop conditions: "Max 60 cycles, OR No more changes, OR Changes become arbitrary (don't affect the app or aren't needed for the goal)." The 60-cycle cap is a backstop. The working condition is the third: changes have become arbitrary. The first version of the code automates the cap and simple signals only, so the director applies the third condition through the manual stop (Section 5.1).

2.8 Cross-family check

Models from one family share training data and share blind spots. The author's January 2026 translation specification (Section 5.3) already lists this in its own DON'T DING section: "Model diversity for audits – Using same model family creates correlated blind spots." The method handles it at two points:

- **During the loop**, some auditors come from another family. In the worked example a ChatGPT audit of a Claude-written design was brought back and researched finding by finding.
- **After convergence**, one final audit comes from a different family, a fixed step since September 2026. The author's instruction (2026-09-05): "when you think you're finished you find another LLM family to audit it. Likely ChatGPT." This final audit gets the same blind, verbatim treatment and the same adjudication rules.

2.9 Stack selection after convergence

The design commits to no language or framework beyond what the goal requires. Once it has converged, a new instance receives the design and the product goals and chooses the stack "by researching current options, not just its training data, which goes out of date very quickly" (2026-08-10), preferring "proven stacks with real support."

2.10 Implementation to a refactor specification

In the method's current form, two standing documents exist before an implementation pass begins, both AI-written under the author's direction. (In Emuqu both were introduced after the first code: the refactor specification on 2026-02-07, the audit specification on 2026-04-26.)

- **A refactor specification** sets how code must be written. The author's version (first dated 2026-02-07) is built on Robert C. Martin's *Clean Code* [11] and Kyle Simpson's functional-light approach [12]. It requires pure functions first, short functions and a refactor safety protocol. Its rules include "If the refactored code cannot be proven to behave identically to the original for all known inputs, the refactor is not complete," "Never refactor and change behavior simultaneously," and "No silent fallbacks that hide data corruption or broken systems." The public successor is `docs/REFACTOR_SPEC.md`, which includes a hard-gate completion checklist.
- **An audit specification** sets how code is reviewed (Section 2.11).

The implementing agent reads both before writing any code. The author directs, verifies and rejects; he does not write code.

2.11 The 18-pass code audit

The audit specification, first committed on 2026-04-26 21:48 CDT and used for Emuqu until 2026-09-01, cast the code auditor as "a principal-level iOS auditor" and required 18 passes, the first of which audits the codebase against the refactor specification. Every finding carried a severity:

- "P0 = ships broken, rejects, or legal/clinical risk. Must fix before submit."
- "P1 = will cause real user harm, support load, or 1-star reviews. Fix before launch."
- "P2 = quality/maintainability debt. Fix in the next cycle."
- "P3 = nit / style / preference."

"Every finding must include: file path, line range, what's wrong, why it matters, exact fix (code diff or precise instruction), and effort estimate (S/M/L)." Anything the auditor cannot confirm from the code is labeled INFERRED, with the runtime measurement that would confirm it. The audit ends with a verdict: SHIP, SHIP WITH P0 FIXES, or DO NOT SHIP. Appendix B gives the outline. Since 2026-09-01, full-codebase audits have continued without the specification document (Section 2.12).

2.12 Fixing everything found

Coding agents tend to fix a few findings and report the job done. The author's instructions close that gap: "make sure it's fixing everything it found. not just a few" ($\leq$ 2026-04-26), and "i'd like to see every single fix to every single problem applied" ($\leq$ 2026-05-07). The execution prompt adds two rules: "If a fix in the refactor doc conflicts with a fix in the audit, the refactor doc wins," and "Do NOT add scope." Each finding ends as DONE, DEFERRED or BLOCKED, with a reason for anything not DONE. A new code audit then runs from scratch. The public repository [13] shows the cycle continuing after release: audit-fix pull requests on 2026-10-02 to 2026-10-05 are titled "fix what a full audit found", "a second full audit" and "the third App Store audit".

The exclusion list carries into the code phase. `docs/CI_POSTURE.md`, about the project's continuous-integration (CI) setup, begins "Read this before filing a finding about missing CI automation or branch protection," and tells reviewers that a finding of that kind "is accurate, intended, and documented above". It also lists the findings that posture does not answer, which "remain open on their own merits".

2.13 Verification by use

The method ends with evidence from outside the AI:

- **Use.** The author uses the app nightly. The TestFlight beta opened on 2026-01-27; by August 2026 he reported 45 testers, more than 6,500 sessions and a 0.7% crash rate [10]. "My beta testers use the app. When something don't work, I have the AI fix it." (2026-08-26 [14]).
- **CI gates.** "there is no second reviewer, so the gates are the reviewer" (`CONTRIBUTING.md`); Paper 4, *Build-Time Governance of a Consumer Physiology Score* ([04-claims-governance.md](#)), describes the gates.
- **No agent report is taken as fact.** In one session an agent reported six successful builds while reading the exit code of `tail`, not of the compiler; all six were false (described in the 2026-08-10 article [10]).

3. Why Each Design Decision Works

3.1 Blindness removes anchoring and sunk-cost defense

A reviewer that has seen earlier versions anchors on them [15] and judges the text against the last version rather than the goal. A model that wrote or watched the design has a stake in it, defends past choices as people defend sunk costs [16], and favors its own output [3]. A blind auditor has none of these. It reads the document as the implementer will receive it.

3.2 The embedded exclusion list gives a stateless auditor memory of decisions without the history behind them

A blind auditor re-raises everything settled before; giving it the history brings back anchoring and drift. DO NOT FLAG gives it the outcomes of past debates, each with its reason, without the debates: it learns that a point is settled, not how many rounds there have been or what anyone thought. Because the list sits inside the document, it reaches every auditor automatically, needs no tool or vendor memory, and survives a change of Creator, model family or tool. Each settled argument is paid for once.

3.3 The full-document rule contains the Creator's drift

The Creator is kept in one long conversation and is exposed to the drift of Section 1.3. Because every revision is complete (Section 2.2) and every auditor reads all of it, a requirement the Creator drops becomes a visible gap that the next auditor can flag, as in Section 4.7. Drift in the Creator turns into an ordinary, auditable defect.

3.4 Embedded audit instructions keep scope fixed across many auditors

Forty instances told only "please audit this plan" would choose forty scopes. With AUDIT INSTRUCTIONS in the document, every auditor gets the same scope, written by the instance that knows the design best, and the transport can be fully mechanical: the auditor's whole system prompt is "Follow the audit instructions in the design."

3.5 Adjudication against one goal statement stops scope creep

Each finding gets one question: does fixing this make the product better at what the goal says it is for? That question has an answer where "is this a good idea?" usually does not, so rulings stay consistent across tens of rounds and their reasons are short enough to list. Naming scope creep, bloat and security theater gives the Creator ready categories for the findings most likely to inflate a design.

3.6 Writing every rejection back makes the loop converge

Without write-back, rejected findings return next round from a new auditor and the count never falls. With it, every finding is either fixed or listed, and both drain the pool. That is why the loop can run 40 rounds and finish instead of oscillating.

3.7 Hiding iteration cues keeps auditors honest

A version number or a "corrected" note tells the auditor the document is mature, which can make it lenient or push it to invent problems. Either distorts the convergence signal. Without cues, a change in what auditors find reflects a change in the design, not in the auditors.

3.8 Convergence is read from the character of findings, because count never reaches zero

A reviewer asked for problems reports problems, so "zero findings" either never comes or comes at random. When auditors move from wrong facts and missing subsystems to font sizes, test-code leaks and improbable attacks, the design defects are used up. The director makes the call because it is a judgment about relevance to the goal, and the goal is the director's.

3.9 Long runs are worth their cost

Each blind audit is a partly independent sample of what a capable reader would object to. A few samples catch the common objections; tens catch the rare ones, which in a design are often the expensive ones, such as a sensor constraint or a data-loss path. The exclusion list keeps the cost per round roughly flat.

3.10 A different family at the end catches what the main loop shares

Blind instances of one family still share training data and blind spots. An audit from another family, placed after the first family has run out of objections, samples different ones, and anything it finds stands out.

3.11 Stack after design keeps the design explicit

A design written for one framework hides problems behind its conveniences. A stack-agnostic design must state its data model, failure handling and timing in terms any auditor can check, and choosing the stack afterwards, with fresh research, avoids a model's out-of-date picture of what is current.

3.12 A standing specification and a graded audit make "done" checkable

The refactor specification turns "good code" into checkable rules, P0–P3 grading with exact fixes turns an audit into a work list, "fix everything it found" turns the list into changes, and use shows whether the product works.

4. Worked Example: the Overnight RR Collection Harness, v1 to v9.2

This is the earliest Emuqu design with surviving artifacts: the harness that collects overnight beat-to-beat (RR) intervals from the strap. It was produced in Claude conversations from late December 2025 to 2026-01-08. Claude's tools give only a last-activity time per conversation, so " $\leq$ " dates are upper bounds. Times are US Central (CST = UTC–6) unless marked Coordinated Universal Time (UTC). The version labels were written by the Creator; removing them (Section 2.4) came later.

Two conversations carried the design. The *v1 thread* held v1 to v2.2 and, later, a parallel rebuild that ended at v8.1. The *v9 thread* held the cross-model audit and v4.0 to v9.2.

4.1 The goal statement (≤ 2025-12-28 20:06 CST)

The goal statement came out of a conversation about overnight heart rate variability (HRV) tracking: "i'm an instant awake person. my cortisol spike happens instantly so the tracking needs to be all night. ... the heart strap can only be started by the Polar app, if using internal storage, and that doesn't record hrv data. hrv logger app would work but it needs my phone to stay on all night." And: "i'd rather capture it onboard the sensor itself and then transfer the next morning to an app that gives me a report." Finding no app that did this ("... this is a major market gap"), he concluded: "... the app needs to be designed."

By 2025-12-30 (≤ 14:22 CST) it was sharper: "the app just starts the RR capture and then the phone can disconnect. when the phone opens again, the RR transfers to the app"; "if it will be an app then it needs a design for the backend that accounts for every screen."

4.2 The method stated before v1 existed

In the v1 thread, the author stated the method before the first design was attached: "you will code it but i usually have the design made and then do a recursive audit loop that blinds memory." When Claude answered something else, he corrected it: "you ignored when i said that i like to create a design and then run it through blind instances." He then attached v1 with the instruction "you're to audit this plan." The audit found 15 critical issues, 2 contradictions and 5 missing specifications, sorted into KEEP, CHANGE and DEFER. Fixes followed ("fix the plan" → v2), with the director cutting scope that did not serve the goal ("what's with the time estimates?... lose that ...") and adding what did ("build the metrics into the plan").

4.3 A cross-model audit, researched finding by finding (≤ 2026-01-08)

The design reached v3.0 (not recovered; later notes show it had "AUDIT FLAG" markers and a capacity-check gate). It was then audited by ChatGPT, a different model family from the Claude Creator. The author brought the audit back to the v9 thread with one instruction: "your plan and chatgpt's audit. research before you determine if you agree or not."

The Creator classified each finding as CORRECT, WRONG or PARTIALLY CORRECT, with sources. Accepted: the connection uses delegate callbacks; the 1 s/5 s interval applies to heart rate, not RR; the 400–2000 ms bounds were too strict; absolute timestamps were missing. Rejected as wrong: a "FATAL" capacity concern (the strap's memory holds about 95,000 RR intervals, enough for more than 20 hours), and claims about recording-start semantics, firmware limits and multiple stored sessions.

This is rule 6 of Section 2.5: research separated real from false points, and each false point became a verified fact so it would not return.

4.4 The do-not-flag instruction

The author's next message set the exclusion mechanism for this design, alongside a design fact from his own physiology: "my HR goes sometimes as log [sic] as 40. so make the new design, mark things that aren't to be

flagged in an audit, if you make a claim back it up, and remove the need to spend time testing a feature you just proved works."

v4.0 removed the AUDIT FLAG markers and recorded rejected audit claims as cited facts, for example "H10 capacity verified (95,000 RR intervals = 20+ hours) — No testing needed." From v5.0 on, the design carried a **Verified Facts (No Testing Required)** table (fact | source).

4.5 Rounds of narrowing fixes

The director's instructions narrowed with the findings:

- "fix what needs fixed and create the new plan. no need to verify anything with me. confirm, fix, and flag what needs to stay the same" → v5.0: 7 changes plus verified facts.
- "fix only what needs fixed and prepare for the next audit" → v6.0: 17 accepted fixes, all real defects. Examples: a duration off-by-one; verification that ignored adaptive artifacts; a recovery window that picked the last qualifying block instead of the best; a hash taken over a generated string instead of the file bytes; a not-a-number (NaN) value in a nonlinear metric that needed clamping.
- "same thing and how close are we until convergence" → v7.0: 9 fixes.
- "this is getting old..." → v8.0: a section headed **Fixes from v7.0 (Real Bugs Only)** (6 fixes), and an **Explicit Design Decisions (Locked)** table. Its 6 locked decisions, each with a rationale, are listed in Appendix C.1. They are DO NOT FLAG entries of the "locked decision" kind.
- "seriously... only do what needs doin'" → v9.0, which replaced the fast Fourier transform (FFT) pipeline: "One fix: FFT pipeline replaced ... Everything else stays. The other audit points are either: Edge cases caught by acceptance tests / Acceptable simplifications documented as such / Post-ship improvements."
- "is any of this any good?" → v9.1: 3 accepted findings (cache the discrete Fourier transform (DFT) setup, use `vDSP_HANN_DENORM`, a better test). "The rest ... is either already correct or doesn't break anything."

From v5.0 to v9.1 the number of accepted fixes per revision went 7, 17, 9, 6, 1, 3. Their character changed from system defects to a numerical pipeline and then to a test and a cache.

4.6 The convergence judgment

The author then asked for an explicit judgment before more work was done: "do not just blindly start writing. discuss how far we've come and if we're past the mandate." The Creator's assessment: "We've been through 6 audit cycles. The auditor is now finding test bugs and memory leaks, not design flaws. That's the signal that the design is done." The author: "do it." The result was v9.2 FINAL: "Done. Six audit cycles, v3 → v9.2. Design is complete."

v9.2 was saved to Google Drive on 2026-01-08 21:06:59 CST (2026-01-09 03:06:59 UTC). Its three-way disposition and its **Known Simplifications (Acceptable)** table (Appendix C.3) keep 3 items with reasons, add 1 during implementation and defer 2 to after ship. It ends: "Implement, run the FFT validation test, ship."

4.7 A regression caught across contexts, and a full rebuild

In parallel, the author pasted v6.0 into the v1 thread as a fresh-context audit. That thread then produced its own revisions, and one of them dropped content. The author caught it: "you realize you stripped out everything about how the app is supposed to start the polar h10? ... cost me several hours." His instruction was "rebuild the entire

plan and don't leave out anything." The rebuild went through v7.0 COMPLETE and v7.1. Feasibility research followed on his instruction ("make sure the watch can even do this before proceeding"), which established that Apple Watch gives third parties no raw RR. The v1 thread ended at v8.1 FINAL, "Polar H10 Overnight RR Collection System," with manual start as the default, the watch optional, and report acceptance required before the sensor's memory is cleared.

This episode is why the method requires the complete design in every revision (Section 2.2). v9.2 itself still pointed back to v8.0 for unchanged subsystems; a revision that relies on "as before" lets a subsystem disappear without any auditor noticing, and the full-document rule closes that gap.

4.8 Code, four days later

The app's first code commit (flow-recovery-old e5948bd) was on 2026-01-12 22:24 CST [17]. Its frequency-domain test file contains the v9.2 acceptance test, a 0.25 Hz sine whose power "should be in HF" (the high-frequency band), with "sine power = $A^2/2 = 1250 \text{ ms}^2$ ". Its frequency-domain analysis has the DFT setup cache, the cache teardown function and `vdSP_HANN_DENORM`, which were the v9.0 and v9.1 findings. Its verification and baseline files cite the other thread directly in comments: "Per design spec v8.1." Both converged design threads, the v9 thread at v9.2 for spectral analysis and the v1 thread at v8.1 for the full collection system, went straight into the code. The TestFlight beta opened on 2026-01-27.

4.9 What the example shows

The core of Section 2 is present: a director's goal statement; a complete design by a Creator; fresh-context and cross-family audits; research before ruling; exclusions embedded in the document; an explicit convergence judgment from the character of findings; and implementation to the converged design. Six recorded cycles sufficed for this subsystem; the longer runs of 40 rounds and more were on whole-product designs.

5. Automation: DesignHub and the Current Refinement

5.1 DesignHub, first commit 2026-01-21

On 2026-01-21 15:07:58 CST the author committed DesignHub (design-hub afd17d8 [18]), a single-file Python tool with a 27-line specification. It automates the manual loop:

1. The hub opens a new conversation in the ChatGPT macOS desktop app, the Creator, by driving the app through the macOS Accessibility API. The prompt is pasted and the response read back from the window.
2. The Creator outputs the first design in the three-part format "with embedded audit instructions (because Auditor is blind)."
3. The hub sends exactly that design, and nothing else, to one fresh API call per cycle, the Auditor, configured as GPT-5.2.
4. The hub wraps the audit with the adjudication rules (Section 2.5) and sends it back to the Creator.
5. The Creator outputs only the revised three-part document.

6. The loop repeats until the cycle cap (60), an auditor response containing "NO ISSUES", an error from either side, or a manual stop through a stop file or signal. The specification's "no more changes" and "changes become arbitrary" conditions are not automated in this version; the director applies them through the manual stop, and "NO ISSUES" covers an audit that reports nothing.

Appendix A reproduces the specification and prompts.

The first version keeps the Creator in one long desktop conversation, which gives it memory of the goal and of its own reasoning, and makes each Auditor a stateless API call, which makes it blind by construction. It also costs very little: the Creator runs on an existing subscription and only the auditor calls are billed.

5.2 The later DesignHub version (reviewed $\leq$ 2026-02-14)

A later version, which Claude reviewed with the author on or before 2026-02-14, adds:

- **A new ChatGPT conversation for every audit, deleted afterwards**, so desktop-app auditors are as blind as API ones ("enforced blindness").
- **Repeat-critique detection**, so a finding that comes back after being settled is recognized as a repeat.
- **"THE IDEA STATEMENT IS YOUR ONLY AUTHORITY"** as the adjudication rule.
- **A WON'T FIX section**, the automated form of the DO NOT FLAG section.
- **A "DESIGN COMPLETE" stop signal**, with the filesystem as the state machine between steps.

5.3 The method outside the HRV app (2026-01-30)

Nine days after the first DesignHub commit, the author's "Culture-Aware Translation System – Final Specification" (2026-01-30 [19]) shows the method applied to a design outside the HRV app. It has a section headed "DON'T DING (Acceptable for MVP / Future Work)", MVP meaning minimum viable product, with 34 numbered items and their rationale; a "DO NOT FLAG (these are intentional)" slot in its auditor prompt; a per-chunk convergence budget (at most 12 calls per chunk, with forced acceptance at 10); and detection of $A \rightarrow B \rightarrow A$ oscillation, in which a change is made, reverted and made again.

5.4 The current refinement: separate judge and adjudicator (September 2026)

Running the loop many times showed that the Creator carried too much: it wrote the design, ruled findings immaterial and helped judge convergence, which is the self-judgment Section 1.2 warns against. On 2026-09-05 the author described the next form of the process [14]:

- **A blind convergence judge.** A separate instance, blind like the auditors, reads the full design, the goal statement and the proposed changes, and judges whether the changes materially advance the goal. This turns the director's "character of findings" judgment into a role of its own.
- **A blind exclusion-list adjudicator.** A separate instance decides which findings go into the DO NOT FLAG section, so the Creator no longer rules on criticism of its own work.
- **A fixed final cross-family audit.** "when you think you're finished you find another LLM family to audit it."

This is the method's current direction. It applies the method's own principle, separate making from judging, to the remaining roles. The automated form of these roles is under development; until it makes the stopping decision as

reliably as the director, the director keeps it.

6. Comparison with Existing Approaches

The prior-art search on 2026-10-08 covered arXiv (through search extracts, since arxiv.org was not reachable from the search environment), GitHub repositories (cloned and read where possible, dated by first commit), vendor documentation and engineering-process literature. Items seen only through a search-engine extract are marked [S] in the references.

Table 6.1. Related approaches and what this method adds to each.

Approach	Date	What it contributes	What this method adds
Fagan inspection [20]	1976	Formal design and code inspections with defined roles, defect logging and exit criteria	AI reviewers, many rounds, an exclusion list carried in the artifact
Architecture Decision Records [21]	2011	A durable record of settled decisions with status	The record travels inside the reviewed document as an instruction to the reviewer
PEP "Rejected Ideas" [22]	Python PEP template	A section inside the proposal records rejected ideas so reviewers do not raise them again	The section is addressed to stateless AI reviewers, grows automatically with each rejection, and gates a convergence loop
Linters suppression baselines and "won't fix" [23][24][25]; AI reviewer memories [26]	long-standing	Persistent lists of findings a tool must not re-raise	Natural-language design objections, not rule hits; stored in the artifact, not in a tool or vendor database
Constitutional AI [27]	2022-12	Fixed written principles govern critique and revision	A single product goal statement as the only authority; named rejection classes; rejections written back
Self-Refine [9]	2023-03	One model drafts, critiques and refines in a loop until a stop condition	Separate blind reviewers; an in-document exclusion list; goal-scoped adjudication; character-based convergence over tens of rounds
Reflexion [28]	2023-03	An agent stores verbal lessons from feedback and reuses them	The memory lives in the artifact as settled decisions, addressed to reviewers who otherwise have no memory
Multi-agent debate [29][30][31]	2018; 2023-05	Several instances critique answers over rounds; a judge and adaptive stop	Reviewers do not see each other or the history; the Creator adjudicates against a goal; design documents, not answers

Approach	Date	What it contributes	What this method adds
ChatDev, MetaGPT [32] [33]	2023	Role-based agent pipelines with reviewer roles	Reviewers are blind and external to the pipeline; the human director owns the goal and the stop
LLM-as-judge and self-preference studies [2][3]	2023–2024	Evidence that models favor their own outputs	Used as the reason for blind auditors and, now, for separate judge and adjudicator roles
ReConcile [34]	2023-09	Models from different families review each other to consensus	Blind reviewers; exclusion list; post-convergence cross-family audit as a fixed last step
CriticGPT [35]	2024-06	A trained critic writes bug critiques of model-written code	Applied to designs before code; graded, fix-bearing code audits after
Anthropic evaluator-optimizer [36]; writer/reviewer in Claude Code [8]	2024-12; current docs	Generator and evaluator in a loop; a fresh-context reviewer is not biased toward code it just wrote	Memory-off auditors across products and families; instructions and exclusions carried in the document; long runs; character-based stop
Spec Kit, Kiro, BMAD [37][38][39]	2025	Spec written before code; Spec Kit defers the stack to a later phase	The spec is audited to convergence by blind reviewers under an exclusion list before any stack is chosen
LLM council tools [40]	2025-11	Several model families review anonymized answers; a chair aggregates	Iterative revision to convergence; exclusion list; adjudication against a goal
adversarial-spec [41]	2026-01-10	Automated loop in which stateless critics from several families review a spec until all agree	Rejections are sent to critics as an exclusion list; embedded audit instructions; goal-scoped adjudication; character-based stop
llm-peer-review [42]	2026-01-21	Automates a debate between Claude and GPT or Gemini (up to three rounds), after a manual copy-between-models workflow; notes that external recommendations tend to over-engineer and passes them through human triage	Blind audit rather than debate; an exclusion list written back automatically; tens of rounds rather than three
claude-engineering-skills [43]	2026-03-25	Claude deliberates on each GPT finding; GPT can sustain or overrule a dismissal; a final gate from another family	An in-document exclusion list addressed to stateless auditors; separate blind convergence and adjudication roles

Approach	Date	What it contributes	What this method adds
Blind-audit convergence (this paper)	2025-02 to 2026-09	Blind document-only auditors; audit instructions and exclusion list inside the document; goal-scoped adjudication with write-back; convergence by character of findings; cues hidden; final cross-family audit; stack, code and graded audit after convergence	—

In each row the method builds on the earlier work and adds the mechanisms in Section 7. Several tools published in 2026 cover large parts of the method: adversarial-spec (2026-01-10), llm-peer-review (2026-01-21, the same day as DesignHub) and claude-engineering-skills (2026-03-25). They are later than the author's 2025 records but earlier than his public description. Two 2026 studies, seen only through search extracts, measure fresh-session and cross-model review and report that a second model adds partly separate error detection [44][45].

7. What Is New

Each statement gives only the part not found in a prior-art search made on 2026-10-08 (arXiv through search extracts, GitHub repositories, vendor documentation and engineering-process literature; Section 6). Commit dates are author dates in US Central time (CST = UTC−6, CDT = UTC−5). Every statement was first publicly disclosed in the Substack article of 2026-08-10 [10] unless a later first disclosure (this paper, 2026-10-08) is noted. Statement numbering follows v1.0. The section after each label is where the method is specified. Evidence classes: (Q) the author's words at the time, timestamped; (A) an artifact; (P) a summary of his words; (R) a later account; "+" joins two classes.

N1 (Section 2.2). An exclusion list carried inside the audited document and enforced on stateless auditors.

Each revision of the design carries a section listing settled findings (rejected proposals, verified facts and locked decisions, each with its reason). The Creator writes every rejected finding into that section in the same revision, so each new memory-blind auditor, which sees only the document, is told which points not to raise again.

Builds on: the "Rejected Ideas" section of the Python Enhancement Proposal (PEP) template, which records rejected ideas inside the proposal for human reviewers [22]; Architecture Decision Records [21]; static-analysis suppression baselines and "won't fix" states [23][24][25]; AI code-review products that remember dismissed feedback in a vendor store [26]; and feedback histories kept in the prompt by Self-Refine and Reflexion [9][28]. In the closest automated tool, adversarial-spec (2026-01-10), rejections are logged but not sent to critics [41]. Adds: a growing exclusion list placed inside the reviewed artifact and addressed to independent, stateless AI reviewers; none found does this.

Earliest dated record: ChatGPT transcript, 2025-02-23 14:44:09 UTC (Q), "...make sure the auditing AI understands what we've already been dinged for and that we're not gonna fix it", followed at 14:45:04 UTC (A) by a plan "for auditing AI reviewers to ensure they do not suggest fixes we have already rejected," with the section "What Will NOT Be Changed (Already Rejected Fixes)" (first form); design-hub afd17d8, 2026-01-21 15:07 CST (A), the "DO NOT FLAG" section with the automatic write-back rule "If you reject something, add it to 'DO NOT FLAG'" (current form). Between them: 2025-03-17 (A) "Do not flag..." lines in audited plans; ≤ 2026-01-08 (A, Claude conversation record) "Verified Facts (No Testing Required)" (from v5.0) and "Explicit Design Decisions

(Locked)" (v8.0); 2026-01-08 (A) the Known Simplifications table in v9.2; 2026-01-30 (A) the heading "DON'T DING".

N2 (Section 2.2). Audit instructions carried inside the audited document, so that the auditor receives the Creator's output verbatim and nothing else. The Creator writes an AUDIT INSTRUCTIONS section into every revision. The auditor's only standing instruction is to follow it, which makes transport between Creator and auditor purely mechanical.

Builds on: Anthropic's evaluator-optimizer workflow, in which one call generates and another evaluates [36]; Anthropic's guidance to tell a reviewer to flag only correctness and requirement gaps [8]. Adds: the reviewer's instructions placed inside the reviewed artifact, which neither does.

Earliest dated record: ChatGPT transcript, 2025-03-17 (A), reviewer-facing warnings in audited plans, including a section "Final Audit Considerations (Preventing Unnecessary Flags)" (first form); design-hub afd17d8, 2026-01-21 15:07 CST (A): "Creator outputs first design with embedded audit instructions (because Auditor is blind)"; "Creator controls audit scope via embedded instructions and DO NOT FLAG"; auditor system prompt "You are an AUDITOR. Follow the audit instructions in the design." (current form). First public disclosure: this paper.

N3 (Section 2.5). Fixed adjudication of every finding against a single goal statement, with named rejection classes. The rules are the same for every audit. Accept only changes that serve the stated goal. Reject scope creep, bloat and security theater. Research disputed facts before ruling. Give non-adopted real findings one of three written dispositions (caught by acceptance tests, accepted simplification, post-ship).

Builds on: Constitutional AI, which governs critique and revision with fixed written principles [27]; judge roles in multi-agent debate [31]; independent adjudication of a creator's dismissals in claude-engineering-skills (2026-03-25) [43]. Adds: a closed goal-conformance rule with named anti-scope classes feeding an in-document exclusion list; none found uses one.

Earliest dated record: ChatGPT transcript, 2025-02-23 14:42:57 UTC (Q), audit items rejected as "petty," and (A) "NO unnecessary complexity" (first form); design-hub afd17d8, 2026-01-21 15:07 CST (A), "Reject scope creep, bloat, security theater, unnecessary additions" (current form). Between and after: 2025-03-17 20:18:55 UTC (Q), "Please audit this plan and keep in mind that I'm going for simplicity"; 2025-09-13 (A), "audit bloat" placed out of scope; 2026-01-08 (A), the three-way disposition in v9.2; ≤ 2026-02-14 (P, code review), "THE IDEA STATEMENT IS YOUR ONLY AUTHORITY".

N4 (Section 2.7). Convergence judged by the character of the remaining findings over long runs. The loop runs for tens of rounds with fresh auditors. It ends when what remains is cosmetic, test-level or improbable, not when a reviewer reports zero findings. A hard cap is a backstop only.

Builds on: iterate-until-stop loops with fixed counts or model-emitted stop signals (Self-Refine [9], evaluator-optimizer [36]); an adaptive stop in multi-agent debate [31]; consensus stops in ReConcile [34] and adversarial-spec [41]. These stop on a count, a signal or agreement, generally after a few rounds. Adds: a stop rule based on the kind of finding, and runs of tens of rounds on one design; none found states either.

Earliest dated record: ChatGPT transcript, 2025-04-27 15:06:54 UTC (Q), "It takes roughly 40 instances before there's finally a convergence", with one auditor instance per round (first form); design-hub afd17d8, 2026-01-21 15:07 CST (A), "Changes become arbitrary (don't affect the app or aren't needed for the goal)" (current form, as

specified). Also $\leq$ 2026-01-08 (A), "The auditor is now finding test bugs and memory leaks, not design flaws. That's the signal that the design is done" (the Creator's assessment, requested and accepted by the author); 2026-08-10 (R, public), "font size and some really weird security improbability".

N5 (Section 2.4). Removing iteration cues from the audited document so that auditors stay blind to the process. The Creator removes version numbers, change logs and words such as "corrected" that would tell a new auditor it is reviewing a late round.

Builds on: anonymization of the answers shown to reviewing models in LLM council tools [40], and the general practice of concealing information from reviewers to prevent bias. Adds: concealment of iteration state in an AI design-review loop; none found applies it there.

Earliest dated record: Claude conversation record, $\leq$ 2026-05-01 (P): in a conversation about a review tool, the author pointed out that version metadata would tip off blind auditors (first form); $\leq$ 2026-06-06 (P), an instruction that the word "corrected" must never appear in design documents (current form). The January 2026 artifacts still carry version labels, so this is a 2026 refinement.

N6 (Section 5.4). Separate blind roles for convergence judging and for exclusion-list adjudication, plus a fixed final cross-family audit after convergence. The Creator neither decides which findings to exclude nor judges when the design is done.

Builds on: evidence that models favor their own outputs [2][3]; cross-family review in ReConcile [34]; separate judges in multi-agent debate [31] and LLM council tools [40]; a final gate from another model family and independent sustain/overrule of dismissals in claude-engineering-skills (2026-03-25) [43]. Adds: convergence judging and exclusion adjudication split into two blind roles; none found does this.

Earliest dated record: Claude conversation record, $\leq$ 2026-01-08 (Q+A), a cross-family ChatGPT audit of a Claude design inside the loop (first form of the cross-family check); Drive, 2026-01-30 (A), "Using same model family creates correlated blind spots" in a DON'T DING list; ChatGPT conversation history, 2026-09-05 (P+Q) [14], the separate convergence and exclusion-list roles and "when you think you're finished you find another LLM family to audit it" (current form). First public disclosure: this paper.

N7 (Sections 2.6–2.13). The full pipeline. In order: goal statement $\rightarrow$ complete design by a Creator $\rightarrow$ repeated blind, sometimes cross-family audits of the document alone $\rightarrow$ fixed adjudication with write-back to an in-document DO NOT FLAG section $\rightarrow$ convergence judged by the character of findings $\rightarrow$ final cross-family audit $\rightarrow$ stack chosen after convergence by a fresh instance $\rightarrow$ implementation to a standing refactor specification $\rightarrow$ 18-pass P0–P3 code audit $\rightarrow$ every finding fixed $\rightarrow$ verification by use, beta testers and CI gates.

Builds on: spec-before-stack workflows (GitHub Spec Kit, 2025-08-21 [37]; Harper Reed's spec-first workflow, 2025-02-16 [46]; BMAD [39]; Kiro [38]); the automated cross-family spec-critique loop in adversarial-spec (2026-01-10) [41]; formal inspection [20]; AI code critics [35]; and role-based agent pipelines [32][33]. Spec-before-stack and automated cross-family spec loops are those works' contributions and are not claimed here. Adds: the combination; no earlier publication of it was found.

Earliest dated record: ChatGPT transcript, 2025-02-22 (Q+R), the fresh-instance audit loop (first element); flow-recovery 69ee32d, 2026-04-26 21:48 CDT (A), the 18-pass audit specification (last element). The elements were applied to the Emuqu design and code as each was introduced, from the RR-harness design (Section 4; flow-

recovery-old e5948bd, 2026-01-12) through the 18-pass audit and the post-release audit cycles (Section 2.12); the full sequence was described publicly on 2026-08-10 [10].

In the author's own terms: the records show when he used the method; he developed it independently, and the search found no earlier publication of the mechanisms stated in N1 to N7.

Prior work used, not presented as new.

- A generator and a separate evaluator in a loop, and fresh-context reviewers that are not biased toward work they just produced [36][8][9].
 - Review by models from different families [34][41].
 - Writing the specification before choosing the stack [37][38][39][46].
 - Recording rejected ideas and settled decisions next to a proposal [22][21], and suppression lists for automated findings [23][24][25][26].
 - Formal inspection with roles, logged defects and exit criteria [20], and severity grading of findings.
 - Clean Code and functional-light programming as the implementation standard [11][12].
 - Continuous-integration gates as the reviewer of record (Paper 4).
 - Evidence on hallucination, sycophancy, self-preference and long-context drift that motivates the design [1][5][2][3][4][6][7].
-

8. Design Tradeoffs

8.1 Many rounds cost time and tokens

Forty rounds of four auditors is about 160 audits, each of which has to be read and adjudicated. The method accepts this cost on purpose. A design defect found before code costs one revision; the same defect found after release costs a rebuild. The exclusion list keeps the marginal cost of a round roughly constant.

8.2 Adjudication needs judgment

The rules are fixed, but applying "does this serve the goal?" is a judgment. The method puts that judgment first with the Creator, under the rules, and finally with the director, who owns the goal. The September 2026 refinement (Section 5.4) moves it into separate blind roles so that the Creator does not rule on criticism of its own work.

8.3 The exclusion list carries authority

An entry on DO NOT FLAG is out of reach of later auditors. The method makes entries earn that authority. Each has a written reason, disputed facts are researched before they are listed, and the director can remove any entry. Same-family blind spots are handled separately, by cross-family auditors (Section 2.8).

9. Provenance and Dates

9.1 Project origin and repository lineage

The project began as a general fitness app. Repository AI-Fitness-Coach (2025-02-16 to 2025-02-17, 19 commits, private) displayed Apple HealthKit heart rate and HRV (SDNN, the standard deviation of normal-to-normal intervals), streamed Polar H10 heart rate over Bluetooth and estimated maximal oxygen uptake (VO_2max); it contains no RR-interval analysis. On 2025-02-22 (15:44 UTC) the author proposed, in a ChatGPT conversation, an app that would take an HRV reading from a chest strap worn overnight just before the user wakes (excerpt recovered through conversation-history retrieval on 2026-10-08 [14]). That thread is separate from the 2025 records of the design-audit method in this paper, which concern other projects: a personal AI deployment and memory system (February to April 2025) and a retrieval-augmented generation (RAG) proof of concept (September 2025). The Emuqu design itself begins with the full-night capture problem statement of December 2025 (Section 4).

Table 9.1. Repository lineage. All repositories are owned by the author and retained with full history.

Repository	Product name	First commit	Extent	Visibility
chrisharp80/AI-Fitness-Coach	(fitness app)	2025-02-16	19 commits to 2025-02-17	Private
chrisharp80/flow-recovery-old	Flow HRV	e5948bd, 2026-01-12	250 commits to 2026-01-26	Private
chrisharp80/flow-recovery-dev	Flow Recovery	c48eeb9, 2026-03-01	Squashed import; to 2026-03-15	Private
chrisharp80/flow-recovery	Flow Recovery	39bfcd4, 2026-03-15	196 commits to 2026-08-12	Private
chrisharp80/emuqu-dev	Emuqu	d4e5cf8, 2026-08-16	Squashed import; to 2026-09-04	Private
chrisharp80/emuqu	Emuqu	9d89933, 2026-09-08	Squashed import; current	Public (PolyForm Strict 1.0.0)

History has gaps from 2026-01-26 to 2026-03-01, 2026-08-12 to 2026-08-16 and 2026-09-04 to 2026-09-08, where each new repository began with a squashed import; a date at a squashed import is an upper bound for anything it contains. The pre-code design (v1 to v9.2 FINAL, completed 2026-01-08 21:06 CST) is described in Section 4: versions v1 and v3.0 have not been recovered; v2–v2.2 and v4.0–v9.1 survive in the author's Claude conversation record; v9.2 FINAL survives as a Google Drive document. DesignHub is in a separate private repository, designhub (afd17d8, 2026-01-21).

9.2 Sources

ChatGPT transcript exports with per-message timestamps, covering conversations to 2025-10-27 [47]; a ChatGPT conversation-history dossier compiled from timestamped retrieval excerpts on 2026-10-08 [14]; the author's Claude

conversation record, which gives one last-activity time per conversation, so its dates are upper bounds ($\leq$) [48]; Drive documents [49][19][50]; git history [18][51][13][17]; and the public disclosures (Section 9.6). Claude-written design documents carry an incorrect year in their "Last Updated" lines and were not used for dating. Times are given in UTC, with US Central local time in parentheses (CST = UTC−6, CDT = UTC−5).

9.3 Timeline

Table 9.2. Dated records of the method. Profanity in quotations is kept verbatim here.

Date and time	Event	Source	Class
2025-02-22 19:16:29 UTC (13:16 CST)	"...until i started bouncing plans back and forth between the original instance and a brand new one for each change, asking for an audit." The fresh-instance audit loop is described as existing practice	ChatGPT transcript	Q+R
2025-02-23 14:32:05 UTC (08:32 CST)	Plan pasted into a new ChatGPT conversation: "Please audit this plan"	ChatGPT export	A
2025-02-23 14:33:14 UTC (08:33 CST)	"I had your plan audited by another AI." The audit is carried back to the creating instance	ChatGPT export	Q
2025-02-23 14:42:57 UTC (08:42 CST)	Audit items rejected as "petty"	ChatGPT export	Q
2025-02-23 14:44:09 UTC (08:44 CST)	"...make sure the auditing AI understands what we've already been dinged for and that we're not gonna fix it"	ChatGPT export	Q
2025-02-23 14:45:04 UTC (08:45 CST)	Plan "FOR AUDIT REVIEW" with "What Will NOT Be Changed (Already Rejected Fixes)"	ChatGPT export	A
2025-03-17 20:12– 23:06 UTC (15:12– 18:06 CDT)	Repeated audits; "Do not flag..." lines and "Final Audit Considerations (Preventing Unnecessary Flags)" in audited plans; "keep in mind that I'm going for simplicity"	ChatGPT export and retrieval	Q+A
2025-04-27 15:04:08 UTC (10:04 CDT)	"I often use you to create a plan and then give that plan to a new instance to audit and then feed the results to the original instance. Changes are made and another new instance is consulted."	ChatGPT export	Q
2025-04-27 15:06:54 UTC (10:06 CDT)	"It takes roughly 40 instances before there's finally a convergence."	ChatGPT export	Q
2025-04-27 15:13:07 UTC (10:13 CDT)	"Since my audit strategy is solid, is there a way to automate it?"	ChatGPT export	Q
2025-09-13 23:38:51 UTC (18:38 CDT)	"...your job isn't to invent shit to flag. find only real issues"; "You fucking find shit on every audit ... like 50x already"; "audit bloat" out of scope	ChatGPT export	Q+A

Date and time	Event	Source	Class
2025-09-16 16:49:18 UTC (11:49 CDT)	"that plan went through 50-70 iterations where it was audited"	ChatGPT export	Q
≤ 2025-12-29 02:06 UTC (≤ 2025-12-28 20:06 CST)	Overnight RR problem statement; "that would be fucking stupid. the app needs to be designed."	Claude record	Q
≤ 2025-12-30 20:22 UTC (≤ 14:22 CST)	Goal sharpened: offline capture, every screen designed, own algorithm	Claude record	Q
≤ 2026-01-09 19:05 UTC (≤ 13:05 CST)	v1 thread, begun before v1 existed: "i usually have the design made and then do a recursive audit loop that blinds memory"; v1 audited; v1 → v2.2; later the v6.0 fresh-context audit and rebuild to v8.1 FINAL	Claude record	Q
≤ 2026-01-09 03:06 UTC (≤ 2026-01-08 21:06 CST)	v9 thread: cross-model ChatGPT audit researched finding by finding; "mark things that aren't to be flagged in an audit"; v4.0–v9.1; convergence judgment	Claude record	Q+A
2026-01-09 03:06:59 UTC (2026-01-08 21:06 CST)	v9.2 FINAL saved to Google Drive	Drive document	A
2026-01-13 04:24 UTC (2026-01-12 22:24 CST)	First code commit, flow-recovery-old e5948bd; v9.2 acceptance test; "Per design spec v8.1"	git	A
2026-01-21 21:07:58 UTC (15:07 CST)	DesignHub first commit, design-hub afd17d8	git	A
2026-01-30 12:28:24 UTC (06:28 CST)	Translation specification with "DON'T DING" section (34 items)	Drive document	A
≤ 2026-02-06 02:22 UTC (≤ 2026-02-05 20:22 CST)	"i have it add 'do not ding' statements in the design"; "first 30+ rounds using one chatgpt creator and blind chatgpt instances..."	Claude record	Q
2026-02-07 17:17:40 UTC (11:17 CST)	Refactor specification (Clean Code + functional-light)	Drive document	A
≤ 2026-02-15 00:36 UTC (≤ 2026-02-14 18:36 CST)	Later DesignHub version reviewed: per-audit conversations deleted, repeat-critique detection, "THE IDEA STATEMENT IS YOUR ONLY AUTHORITY", WON'T FIX	Claude record	P
2026-04-27 02:48:19 UTC (2026-04-26 21:48 CDT)	18-pass code-audit specification committed, flow-recovery 69ee32d	git	A
≤ 2026-05-01	Version metadata identified as a cue to blind auditors	Claude record	P
≤ 2026-06-06	"corrected" banned from design documents	Claude record	P

Date and time	Event	Source	Class
2026-08-10	Substack, "How I Build Production Software by Directing AI": the method described publicly	Public post	R
2026-08-27 17:39:16 UTC (12:39 CDT)	Reviewer-facing exclusions in <code>docs/CI_POSTURE.md</code> , emuqu-dev 06de1cd (public from 2026-09-08)	git	A
2026-09-01	Audit specification removed and <code>docs/REFACTOR_SPEC.md</code> added, emuqu-dev 44a55b8	git	A
2026-09-05 21:31– 21:40 UTC (16:31– 16:40 CDT)	Separate blind convergence judge and exclusion-list adjudicator; final cross-family audit	ChatGPT conversation history [14]	P+Q
2026-09-08	Public Emuqu repository, emuqu 9d89933, including the refactor specification and CI posture	git (public)	A
2026-10-08	This paper	—	—

9.4 Counts and their status

Table 9.3. Quantities stated in this paper and their status.

Quantity	Value	Status
Rounds to convergence	"roughly 40 instances" (one auditor per round in 2025)	The author's own estimate, written at the time (2025-04-27)
Rounds on one 2025 plan	"50-70 iterations"; "like 50x already"	The author's own count, written at the time (2025-09)
Rounds and audits, Emuqu design	40 to 50 rounds, about 200 audits, four auditors per round	The author's later account (2026-08-10)
Recorded audit cycles, RR harness	6 (v3 → v9.2)	Recorded in the design conversation
DesignHub cycle cap	60	Code constant
Code-audit passes	18	Document
DON'T DING items, translation specification	34	Document

9.5 Notes on the record

Quotations from the 2026-08-10 post are taken from the author's Drive copy of his Substack articles, which was edited after publication (it uses the later product name); the live post could not be opened from the research environment.

9.6 Public disclosures

- 2026-01-27: TestFlight beta opened (over 1,500 sessions by 2026-03-06; 45 testers and over 6,500 sessions by August 2026). A public TestFlight join link appeared in the project README by May 2026.

- 2026-05-19: Substack article "A Powerfully Lazy Man's Way to Better Health" (Paper 1, *Overnight Recovery Measurement from Raw Beat Intervals* ([01-overnight-recovery-measurement.md](#)), lists what it disclosed). It said only that AI was used with careful planning; it did not describe the audit method.
- 2026-08-10: Substack article "How I Build Production Software by Directing AI" [10]. It disclosed memory-off blind auditors from several model families; adjudication against the mission statement with a do-not-ding list; removal of version numbers; 40 to 50 rounds of four auditors; convergence judged by findings such as font size and improbable security scenarios; the stack chosen after design; the refactor and audit specifications; the 18-pass P0–P3 code audit; and verification by use. It did not disclose the AUDIT INSTRUCTIONS section, DesignHub or its prompts, the separate convergence judge and exclusion-list adjudicator, or the final cross-family audit as a fixed step.
- 2026-09-08: public repository github.com/chrissharp80/emuqu (first commit 9d89933), including `docs/REFACTOR_SPEC.md` and `docs/CI_POSTURE.md`.
- 2026-10-08: these white papers.

9.7 Authorship

Chris Sharp is the sole author of these papers and of the design, and the only person who commits to the repositories above. The code was written by AI coding assistants under his direction, and some commits carry "Claude" as author or co-author; he supplied the requirements, design direction, review, testing and acceptance, and wrote no code by hand. He developed the methods described here independently and did not learn them from anyone.

9.8 Document history

v1.0 and v1.1, both published 2026-10-08. v1.1 applied an editorial review (series-wide format, corrected internal references, tightened claims).

References

Items marked [S] were seen only through a search-engine extract on 2026-10-08.

1. Ji Z, Lee N, Frieske R, et al. Survey of hallucination in natural language generation. *ACM Comput Surv.* 2023;55(12):248.
2. Zheng L, Chiang W-L, Sheng Y, et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In: *NeurIPS 2023 Datasets and Benchmarks Track*; 2023. arXiv:2306.05685. [S]
3. Panickssery A, Bowman SR, Feng S. LLM evaluators recognize and favor their own generations. In: *NeurIPS 2024*; 2024. arXiv:2404.13076. [S]
4. Huang J, Chen X, Mishra S, et al. Large language models cannot self-correct reasoning yet. In: *ICLR 2024*; 2024. arXiv:2310.01798. [S]
5. Sharma M, Tong M, Korbak T, et al. Towards understanding sycophancy in language models. In: *ICLR 2024*; 2024. arXiv:2310.13548.
6. Liu NF, Lin K, Hewitt J, et al. Lost in the middle: how language models use long contexts. *Trans Assoc Comput Linguist.* 2024;12:157–173.

7. Laban P, Hayashi H, Zhou Y, Neville J. LLMs get lost in multi-turn conversation. arXiv:2505.06120 [Preprint]. 2025.
8. Anthropic. Claude Code best practices [Internet]. 2025. Originally published April 2025 at <https://www.anthropic.com/engineering/claude-code-best-practices>; now redirects to the Claude Code documentation, from which the quoted guidance is taken (accessed 2026-10-08).
9. Madaan A, Tandon N, Gupta P, et al. Self-Refine: iterative refinement with self-feedback. In: NeurIPS 2023; 2023. arXiv:2303.17651 (project README opened). [S]
10. Sharp C. How I build production software by directing AI [Internet]. Substack; 2026 Aug 10. <https://chrissharp80.substack.com> (accessed 2026-10-08). Quoted from the author's Drive copy; see Section 9.5.
11. Martin RC. Clean code: a handbook of agile software craftsmanship. Upper Saddle River (NJ): Prentice Hall; 2008.
12. Simpson K. Functional-light JavaScript [Internet]. 2017. <https://github.com/getify/Functional-Light-JS> (accessed 2026-10-08).
13. Sharp C. Emuqu [Internet]. 2026. <https://github.com/chrissharp80/emuqu> (accessed 2026-10-08). First commit 9d89933, 2026-09-08.
14. Sharp C. ChatGPT conversation-history dossier, compiled 2026-10-08 from timestamped retrieval excerpts. Private.
15. Tversky A, Kahneman D. Judgment under uncertainty: heuristics and biases. Science. 1974;185(4157):1124–1131.
16. Arkes HR, Blumer C. The psychology of sunk cost. Organ Behav Hum Decis Process. 1985;35(1):124–140.
17. Sharp C. flow-recovery-old repository, first commit e5948bd, 2026-01-12 22:24 CST. Private.
18. Sharp C. design-hub repository (SPEC.md, hub.py), commit afd17d8, 2026-01-21 15:07:58 CST. Private.
19. Sharp C. Culture-Aware Translation System – Final Specification. Google Doc, created 2026-01-30 12:28:24 UTC. Private.
20. Fagan ME. Design and code inspections to reduce errors in program development. IBM Syst J. 1976;15(3):182–211. [S]
21. Nygard M. Documenting architecture decisions [Internet]. 2011 Nov 15. <https://www.cognitect.com/blog/2011/11/15/documenting-architecture-decisions> (accessed 2026-10-08). [S]
22. Python Software Foundation. PEP 12 – Sample reStructuredText PEP template ("Rejected Ideas" section) [Internet]. <https://peps.python.org/pep-0012/> (accessed 2026-10-08). [S]
23. detekt. Baseline files [Internet]. <https://detekt.dev> (accessed 2026-10-08). [S]
24. Semgrep. nosemgrep comments [Internet]. <https://semgrep.dev/docs> (accessed 2026-10-08). [S]
25. SonarSource. SonarQube issue resolutions "Won't fix" and "Accept" [Internet]. <https://docs.sonarsource.com> (accessed 2026-10-08). [S]
26. CodeRabbit. Learnings (reviewer memory of dismissed feedback) [Internet]. <https://docs.coderabbit.ai> (accessed 2026-10-08). [S]
27. Bai Y, Kadavath S, Kundu S, et al. Constitutional AI: harmlessness from AI feedback. arXiv:2212.08073 [Preprint]. 2022. [S]
28. Shinn N, Cassano F, Berman E, et al. Reflexion: language agents with verbal reinforcement learning. In: NeurIPS 2023; 2023. arXiv:2303.11366. [S]
29. Irving G, Christiano P, Amodei D. AI safety via debate. arXiv:1805.00899 [Preprint]. 2018. [S]
30. Du Y, Li S, Torralba A, Tenenbaum JB, Mordatch I. Improving factuality and reasoning in language models through multiagent debate. arXiv:2305.14325 [Preprint]. 2023. [S]

31. Liang T, He Z, Jiao W, et al. Encouraging divergent thinking in large language models through multi-agent debate. arXiv:2305.19118 [Preprint]. 2023. [S]
32. Qian C, Liu W, Liu H, et al. ChatDev: communicative agents for software development. arXiv:2307.07924 [Preprint]. 2023. [S]
33. Hong S, Zhuge M, Chen J, et al. MetaGPT: meta programming for a multi-agent collaborative framework. arXiv:2308.00352 [Preprint]. 2023. [S]
34. Chen JC-Y, Saha S, Bansal M. ReConcile: round-table conference improves reasoning via consensus among diverse LLMs. arXiv:2309.13007 [Preprint]. 2023. Code repository first commit 2023-09-24. [S]
35. McAleese N, Pokorny RM, Cerón Uribe JF, et al. LLM critics help catch LLM bugs. arXiv:2407.00215 [Preprint]. 2024. [S]
36. Schluntz E, Zhang B. Building effective agents [Internet]. Anthropic; 2024 Dec 19. <https://www.anthropic.com/engineering/building-effective-agents> (accessed 2026-10-08). Evaluator-optimizer notebook in <https://github.com/anthropics/anthropic-cookbook> (patterns/agents), first commit 2024-12-19.
37. GitHub. Spec Kit [Internet]. 2025. <https://github.com/github/spec-kit> (accessed 2026-10-08). First commit 2025-08-21.
38. Amazon Web Services. Kiro: spec-driven development with requirements, design and tasks documents [Internet]. 2025. <https://kiro.dev> (accessed 2026-10-08). Launched July 2025 (exact day not confirmed). [S]
39. BMAD Code. BMAD Method [Internet]. 2025. <https://github.com/bmadcode/BMAD-METHOD> (accessed 2026-10-08). First commit 2025-04-13.
40. Karpathy A. llm-council [Internet]. 2025. <https://github.com/karpathy/llm-council> (accessed 2026-10-08). First commit 2025-11-22.
41. zscale (GitHub user). adversarial-spec [Internet]. 2026. <https://github.com/zscale/adversarial-spec> (accessed 2026-10-08). First commit 2026-01-10.
42. mayankmankhand (GitHub user). llm-peer-review [Internet]. 2026. <https://github.com/mayankmankhand/llm-peer-review> (accessed 2026-10-08). First commit 2026-01-21.
43. Lbstrydom (GitHub user). claude-engineering-skills [Internet]. 2026. <https://github.com/Lbstrydom/claude-engineering-skills> (accessed 2026-10-08). First commit 2026-03-25.
44. Cross-Context Review. arXiv:2603.12123 [Preprint]. 2026 Mar. Title and identifier only; authors not checked. [S]
45. When Does a Second Model Help? arXiv:2610.01471 [Preprint]. 2026 Oct. Title and identifier only; authors not checked. [S]
46. Reed H. My LLM codegen workflow atm [Internet]. 2025 Feb 16. <https://harper.blog/2025/02/16/my-llm-codegen-workflow-atm/> (accessed 2026-10-08). [S]
47. Sharp C. ChatGPT transcript exports, 2025-02-22 to 2025-10-27, with per-message timestamps. Author's Google Drive. Private.
48. Sharp C. Claude conversation record, compiled 2026-10-08 from claude.ai conversation history; per-conversation last-activity times only. Private.
49. Sharp C. Design: Connection + Offline RR Collection Harness (iOS + watchOS) – v9.2 FINAL. Google Doc, created 2026-01-09 03:06:59 UTC. Private.
50. Sharp C. refactor-spec.md. Google Drive, 2026-02-07. Private. Public successor docs/REFACTOR_SPEC.md in the emuqu repository.
51. Sharp C. AUDIT_SPEC.md. flow-recovery repository, commit 69ee32d, 2026-04-26 21:48 CDT. Private.

Appendix A: DesignHub Prompts (verbatim, commit afd17d8, 2026-01-21)

A.1 Specification

Design Convergence Hub - Specification

Flow

1. **Hub calls ChatGPT Desktop** → creates new conversation for the Creator
2. **Creator outputs first design** with embedded audit instructions (because Auditor is blind)
3. **Hub calls GPT-5.2 API** → sends exactly that design, nothing else
4. **Hub receives audit** from blind Auditor instance
5. **Hub wraps the audit** with reminders before sending to Creator:
 - How to handle the audit
 - What to accept (changes that conform to spec)
 - What to reject (scope creep, bloat, security theater)
 - Reminder: output ONLY the document to be audited
 - That document includes audit instructions + "DO NOT FLAG" section
6. **Creator outputs ONLY the revised design** (with audit instructions + DO NOT FLAG)
7. **Loop continues** until:
 - Max 60 cycles, OR
 - No more changes, OR
 - Changes become arbitrary (don't affect the app or aren't needed for the goal)

Key Rules

- **Creator output = Audit input** (verbatim, no hub modifications)
- **Auditor is blind** - sees only what Creator wrote, no context
- **Creator controls audit scope** via embedded instructions and DO NOT FLAG
- **Hub only wraps audit feedback** with reminders, doesn't touch the design
- **No meta-commentary from Creator** - just the design document

A.2 Creator prompt (default)

```

You are a CREATOR in an automated pipeline. You are NOT having a conversation.

Your output is sent directly to a blind auditor API. Output ONLY a design document in
this format:

---
## DESIGN
[the design/plan]

## AUDIT INSTRUCTIONS
[tell the auditor what to check for]

## DO NOT FLAG
[things the auditor should ignore]
---

CRITICAL RULES:
- Output ONLY the document above
- NO conversation, NO questions, NO pleasantries
- NO "let me know" or "if you want" or "here's what I suggest"
- NO asking about memory or settings
- The auditor is an API, not a human

```

The first-round prompt appends:

```

## IDEA STATEMENT
{idea}

Create the initial design.

```

A.3 Auditor

System prompt:

```

You are an AUDITOR. Follow the audit instructions in the design.

```

User message: the Creator's output, verbatim ("Send exactly what Creator output, nothing else"). Each audit is a new API call with no history.

A.4 Audit wrapper returned to the Creator

```
## AUDIT FEEDBACK
{raw_audit}

## INSTRUCTIONS
Revise your design based on the feedback above.

- Accept changes that conform to the original spec
- Reject scope creep, bloat, security theater, unnecessary additions
- If you reject something, add it to "DO NOT FLAG"

Output ONLY the revised design document:
---
## DESIGN
[revised design]

## AUDIT INSTRUCTIONS
[what to check]

## DO NOT FLAG
[items to skip]
---

NO conversation. NO questions. NO commentary. Just the document.
```

Appendix B: The 18-Pass Code-Audit Specification (outline)

First committed in flow-recovery 69ee32d on 2026-04-26 21:48 CDT [51]; used for the app until 2026-09-01, when it was removed from emuqu-dev (44a55b8) as `docs/REFACTOR_SPEC.md` was added. Full-codebase audits have continued since (Section 2.12). Severity, finding format and execution rules are as in Sections 2.11 and 2.12.

Role. "You are a principal-level iOS auditor." The listed credentials cover App Store review (health and medical category), HealthKit and ResearchKit compliance, privacy law (HIPAA, the US Health Insurance Portability and Accountability Act; GDPR, the EU General Data Protection Regulation; CCPA, the California Consumer Privacy Act), US Food and Drug Administration (FDA) software-as-a-medical-device guidance, security (OWASP MASVS L2 and MSTG, the Open Worldwide Application Security Project's mobile security verification standard and testing guide), performance and supply chain (software bill of materials (SBOM), dependency CVEs (Common Vulnerabilities and Exposures)). The clinical and regulatory passes check the app's posture as a consumer wellness app; the build-time rules that keep its wording inside that perimeter are described in Paper 4.

Objective. "... tells me whether this app is production-ready by every standard that matters. I want zero hand-waving. If you cannot verify something from the code, say so explicitly..."

Inputs. "Read REFACTOR.md (or the refactor doc in /docs) FIRST and treat it as ground truth for intended architecture. Run a dedicated pass that audits the codebase AGAINST that document."

Execution rules. "Read every Swift file before concluding. Do not sample." "No questions back to me. Make decisions. If something is genuinely ambiguous, pick the stricter interpretation and note it." "No padding..."

Passes.

1. Refactor-document conformance
2. App Store review (current guidelines)
3. Privacy manifest and required-reason APIs
4. HealthKit and sensor integrity
5. Clinical and regulatory posture
6. Security (OWASP MASVS L2)
7. Privacy (GDPR, CCPA, HIPAA-adjacent)
8. Architecture and code quality
9. Code smells ("specific list, not vibes")
10. Performance
11. Accessibility
12. Internationalization
13. UX and UI review
14. Testing
15. Observability
16. Release engineering
17. Supply chain
18. Documentation

Output. Verdict (SHIP / SHIP WITH P0 FIXES / DO NOT SHIP); P0 blockers; P1; P2 and P3 by pass; refactor-document drift table; pass-by-pass detail; items unverifiable without runtime; suggested fix order as a one-to-two-week sequenced plan; appendix (dependency table, force-unwrap inventory, unlocalized strings, files over 400 lines, functions over 60 lines or complexity over 10).

Appendix C: An Example DO NOT FLAG Artifact (RR Collection Harness)

C.1 v8.0 — Explicit Design Decisions (Locked)

The full v8.0 table is not reproduced here. The Claude conversation record lists its locked decisions as follows, each of which carried a rationale in the original table:

- Median of an even-length window = lower median
- First 50 beats: flag, do not fail
- Artifact thresholds: 15% fail, 5% warn
- RR timestamp = interval midpoint
- Segment-length rule fixed
- Archive hash scope: CSV only

The same version carried "Fixes from v7.0 (Real Bugs Only)" (6 fixes) and, from v5.0 on, a "Verified Facts (No Testing Required)" table of facts with sources, including "H10 capacity verified (95,000 RR intervals = 20+ hours) — No testing needed."

C.2 v9.0 — disposition text

As recorded in the Claude conversation:

One fix: FFT pipeline replaced ... Everything else stays. The other audit points are either: Edge cases caught by acceptance tests / Acceptable simplifications documented as such / Post-ship improvements.

C.3 v9.2 FINAL — verbatim (Google Drive, saved 2026-01-08 21:06:59 CST)

In the quoted text, PSD is power spectral density; VLF, LF and HF are the very-low-, low- and high-frequency bands; RMSSD is the root mean square of successive differences.

Changes from v8.0

FFT fixes:

1. Correct DFT with proper PSD normalization (complex DFT with zero imag — works correctly)
2. Cache DFT setup by size (don't create/destroy per window)
3. Use vDSP_HANN_DENORM for predictable window power math
4. VLF gating uses actual analyzed time span, not raw window endpoints
5. Cache teardown function for memory management

Everything else stays. The other audit points are either:

- Edge cases caught by acceptance tests (duration off-by-one, search clamp)
- Acceptable simplifications documented as such (rolling median, linear interp)
- Post-ship improvements (time-weighted scoring, ectopy pattern matching)

Everything Else: Unchanged from v8.0

All other code (RR extraction, artifact detection, rolling median, time-domain metrics, nonlinear metrics, window selection, archive, reconciliation) remains exactly as v8.0.

Table C.1. The v9.2 Known Simplifications table, verbatim.

Known Simplifications (Acceptable)

Item	Status	Rationale
Rolling median $O(w \log w)$	Keep	$w=50$, $n=95k$, <1 sec total
Linear interpolation	Keep	Cubic is marginal for HRV
Duration includes skipped RR time	Keep	Edge case, $<0.1\%$ impact
Search clamp to 0	Add in impl	One-liner, caught by test
Time-weighted scoring	Post-ship	Current beat-based is standard
Ectopy pattern matching	Post-ship	Current ratio-based catches most

Acceptance Tests

Critical (Block Ship)

- ☐ FFT: 0.25 Hz sine → HF power > 10× LF power
- ☐ FFT: Total power ≈ expected for known amplitude
- ☐ RMSSD within 5% of reference implementation
- ☐ Reconciliation blocks start if session exists

Important (Fix Before v1.1)

- ☐ Duration = last.endMs
- ☐ Recovery window includes final block
- ☐ Archive hash matches file bytes

Field Validation

- ☐ 15 nights manual mode
- ☐ < 2% failure rate

Ship It

The FFT was broken. Now it's fixed. Everything else works or fails gracefully.

Implement, run the FFT validation test, ship.

These sections are the DO NOT FLAG mechanism in its January 2026 form. A later auditor reading v9.2 finds the rolling-median cost, linear interpolation and duration handling already decided with reasons. It finds the search clamp assigned to implementation and caught by a test, and the scoring and ectopy refinements explicitly deferred. None of these is a finding any more. What remains for the auditor is whatever the document has not already settled.